Shared Household Journal — Project Knowledge Base

Claude Code in VS Code Edition | Repository-aware project reference | Draft v2

Purpose

This knowledge base is a compact reference for the product and its implementation workflow. It is intended to be read alongside the PRD and TRD. It must be updated when verified project decisions change. Do not treat unconfirmed assumptions as settled facts.

Product identity

Name: Shared Household Journal.

Purpose: help people sharing a home manage shared costs and home coordination in a transparent, friendly way.

Experience: warm, calm, practical, human, and home-oriented.

Primary users: roommates and other shared households.

Product boundary: record and explain shared finances; it is not a bank and does not move money.

Feature map

Household and membership; expenses; equal/percentage/custom splits; derived balances; settlement records; activity journal; recurring bills; shopping list; chores; budgets and summaries; notification preferences; profile/settings. Optional later work includes receipt OCR, assisted categorization, debt simplification, and export. Optional features require explicit prioritization and privacy review.

Visual language

Palette: #4F7CAC primary; #7FB069 secondary; #F4A259 accent; #FAF8F5 page background; #FFFFFF surfaces; #2D3748 main text; #22C55E success; #F59E0B warning; #EF4444 danger. Suggested font: Plus Jakarta Sans or Inter. Apply only after auditing existing tokens/components. Prefer clear hierarchy, friendly labels, restrained color, and responsive layouts.

Domain glossary

Household: shared workspace and authorization boundary.

Member: authenticated user associated with a household.

Expense: recorded cost with payer, amount, date, category, and split participants.

Split: allocation of an expense among participants.

Balance: net obligation derived from expenses and settlements.

Settlement: an in-app record that an obligation was settled; it does not prove or initiate an external transfer.

Recurring bill: expense template or schedule for future expected costs.

Journal event: meaningful action recorded for household context.

RLS: database Row Level Security policies that constrain data access.

Important product rules

1. No subscription/Stripe billing is part of the product baseline.

2. Do not claim the app transfers money.

3. Expense split totals must reconcile to the original amount.

4. Balances must be explainable from underlying expenses and settlements.

5. Household data must be isolated from non-members.

6. Manual correction must remain possible for optional AI/OCR suggestions.

7. No fake demo data should be presented as live user data without clear labeling.

8. Existing repository implementation and verified decisions take precedence over assumptions in this reference.

Repository facts — fill after audit

Do not guess these values. Claude Code should populate this section after inspecting the workspace:

• Framework/version:

• Package manager and lockfile:

• Main source directory:

• Route structure:

• Database/auth provider and client setup:

• Current schema/migration location:

• Existing UI library and design tokens:

• Test/lint/typecheck/build commands:

• Current implemented features:

• Known failures or blockers:

• Deployment target:

• Environment variable NAMES (never values):

Decision log

Record decisions with date, status, rationale, and impact. Suggested statuses: Confirmed, Proposed, Open, Superseded.

Open decisions include currency/multi-currency policy, invitation expiry, deletion policy, settlement reversal, recurring-bill timezone, notification channels, export/retention, OCR/AI provider, and deployment target.

Never silently convert a proposal into a confirmed decision.

Claude Code in VS Code conventions

• Start with repository audit; do not implement blindly.

• Work in bounded phases and tasks.

• Preserve user changes and existing working code.

• Review diffs in VS Code before accepting.

• Never expose secrets or print environment values.

• Never perform destructive migrations without explicit approval.

• Use actual repository scripts and report the commands that were run.

• Do not fabricate test results, features, or completion.

• Update docs when a verified architecture or product decision changes.

• Keep CLAUDE.md concise; use it for project rules and commands, not as a duplicate of all project documents.

Recommended phase sequence

0. Repository discovery and baseline.

1. UI foundation and responsive shell.
2. Authentication and household membership.
3. Expenses and split engine.
4. Balances and settlement records.
5. Journal and notifications.
6. Recurring bills.
7. Shopping list and chores.
8. Analytics and budgets.
9. Optional OCR/AI capabilities.
10. Security, accessibility, testing, and release hardening.

Task completion checklist

Before calling a task complete, verify:

■ Existing behavior and working files were preserved.

■ Requirements and assumptions were reviewed.

■ Relevant code changes are scoped and understandable.

■ Input validation and authorization are considered.

■ Tests/checks were run where available.

■ Results are reported honestly; unrun checks are disclosed.

■ Migration and environment changes are documented.

■ Documentation and decision log are updated when needed.

■ Next step is identified.